과제 4 요구사항 요약

# 쿠폰 사용과 회의실 예약

성격이 다른 두 가지 동시성 문제를 구현하며, 락을 어느 범위까지 걸어야 하는지 연습하는 워밍업 과제입니다.

- 이 문서는 과제의 전체 흐름과 주요 요구사항을 정리한 **요약 자료**입니다.
- 이 과제는 과제 1~3보다 짧고 단순한 **워밍업** 과제입니다.
- API 규격과 힌트는 저장소의 `README.md` 를 기준으로 확인해 주세요.

**2개**
**서로 독립된 기능**
쿠폰과 회의실 예약은 서로 영향을 주지 않습니다

**4개**
**구현할 TODO**
쿠폰 발급, 쿠폰 사용, 예약 생성, 사용자별 예약 조회

**11개**
**점검 시나리오**
모든 시나리오가 기대 결과대로 동작해야 합니다

**제한 없음**
**소요 시간**
1~1.5시간 분량입니다

## 전체 구조




파란색은 직접 구현할 부분, 회색은 수정할 수 없는 저장소입니다. 두 기능은 코드와 데이터를 공유하지 않습니다.

## 과제 목표와 구현 범위

### 과제 목표

- 같은 쿠폰이 **두 번 사용되지 않게** 합니다.
- 같은 회의실에 **시간이 겹치는 예약이 두 건 이상 성공하지 않게** 합니다.
- 다른 사용자나 다른 회의실의 요청은 **기다리지 않고** 처리되게 합니다.

### 구현하지 않아도 되는 것

- DB, Docker, 외부 서버 (모두 메모리에 저장)
- 긴 설계 문서 (README에 3~4줄로 작성)
- 테스트 코드 (구현만 제출)

## 가장 놓치기 쉬운 상황: 확인과 저장 사이에 다른 요청이 끼어드는 경우

- 쿠폰과 예약 모두 **확인한 뒤 저장하는** 구조라서 같은 문제가 생길 수 있습니다.
- 다만 하나만 처리되어야 하는 대상이 다릅니다. 쿠폰은 쿠폰 한 장이고, 예약은 시간 구간입니다.

**1. 확인**
보유 쿠폰 중 할인액이 가장 큰 쿠폰을 찾거나, 회의실의 기존 예약과 겹치는지 확인합니다.

→

**2. 다른 요청**
그 사이 다른 요청이 같은 사용자나 같은 회의실을 동시에 확인합니다.

→

**3. 중복 처리**
두 요청이 모두 통과해 쿠폰이 두 번 사용되거나, 겹치는 예약이 함께 성공합니다.

→

**4. 해결 방향**
쿠폰은 **사용자 단위**로, 예약은 **회의실 단위**로 확인과 저장을 하나로 묶습니다.

전체를 하나의 락으로 잠그지 않고 범위를 좁혀야 하는 이유는 README.md 10장 힌트 1을 참고해 주세요.

## 반드시 지켜야 할 API 명세

| 항목 | 규칙 |
|---|---|
| 쿠폰 발급 | `POST /api/users/{userId}/coupons`<br>성공 **204**, 잘못된 type **400** |
| 쿠폰 사용 | `POST /api/users/{userId}/coupons/use`<br>할인액이 가장 큰 쿠폰을 자동으로 선택합니다. 성공 **200**, 사용할 쿠폰 없음 **404** |
| 예약 생성 | `POST /api/reservations`<br>성공 **201**, 시간 겹침 **409**, 입력 오류 **400** |
| 예약 조회 | 회의실별 조회 `GET /api/reservations/rooms/{roomId}` (제공됨)<br>사용자별 조회 `GET /api/reservations/users/{userId}` (직접 구현) |

## 점검 시나리오 11개

### 1. 쿠폰 (S1~S5) 5개

- 발급 후 사용: 할인액이 가장 큰 쿠폰 적용
- 2장 보유, 동시 사용 2건: 둘 다 성공
- 1장 보유, 동시 사용 5건: 1건만 성공
- 쿠폰이 없을 때 사용: 404
- 다른 사용자의 동시 사용: 서로 기다리지 않고 처리

### 2. 회의실 예약 (S6~S11) 6개

- 정상 예약: 201
- 같은 회의실, 겹치는 시간(순차): 409
- 같은 회의실, 겹치는 시간 동시 5건: 1건만 성공
- 같은 회의실, 끝과 시작만 맞닿음: 둘 다 성공
- 다른 회의실, 같은 시간 동시 예약: 둘 다 성공
- 사용자별 조회: 해당 사용자의 예약만 조회

## 권장 진행 순서

1. **이 문서로 과제의 전체 흐름을 확인합니다.**
   쿠폰과 예약은 서로 독립되어 있으므로 어느 쪽부터 시작해도 됩니다.
2. **README.md를 처음부터 끝까지 읽습니다.**
   API 명세와 시나리오를 먼저 확인해 주세요.
3. **`./gradlew bootRun` 으로 시작 프로젝트를 실행합니다.**
   제공된 조회 API(GET)가 동작하는지 먼저 확인해 주세요.
4. **쿠폰 발급 TODO부터 구현합니다.**
   가장 단순한 TODO를 먼저 구현한 뒤, 동시성 처리가 필요한 TODO로 넘어가는 것을 권장합니다.
5. **구현이 막히면 README 10장의 힌트를 하나씩 확인합니다.**

**정해진 정답이 있는 과제가 아닙니다.** 락을 어디에 어떻게 걸었는지, 그 이유를 README에 작성해 주세요.
세부 요구사항은 저장소의 **README.md**를 기준으로 확인해 주세요.